

## #TASK 2.4: Creating Callbacks

```

# Define the callback function to update the input container based on the selected statistics
@app.callback(
    Output(component_id='select-year', component_property='disabled'),
    Input(component_id='dropdown-statistics', component_property='value')
)
def update_input_container(selected_statistics):
    if selected_statistics == 'Yearly Statistics':
        return False
    else:
        return True

#Callback for plotting
# Define the callback function to update the input container based on the selected statistics
@app.callback(
    Output(component_id='output-container', component_property='children'),
    [Input(component_id='dropdown-statistics', component_property='value'),
     Input(component_id='select-year', component_property='value')],
)
def update_output_container(selected_statistics, selected_year):
    print(selected_year, selected_statistics)
    if selected_statistics == 'Recession Period Statistics':
        # Filter the data for recession periods
        recession_data = data[data['Recession'] == 1]
        #TASK 2.5: Create and display graphs for Recession Report Statistics
        #Plot 1 Automobile sales fluctuate over Recession Period (year wise)
        # use groupby to create relevant data for plotting
        yearly_rec=recession_data.groupby('Year')['Automobile_Sales'].mean().reset_index()
        R_chart1 = dcc.Graph(
            figure=px.line(data_frame=yearly_rec,
                           x='Year',
                           y='Automobile_Sales',
                           title="The average monthly sales for each year during periods of recession"))
        #Plot 2 Calculate the average number of vehicles sold by vehicle type
        # use groupby to create relevant data for plotting
        #Hint:Use Vehicle_Type and Automobile_Sales columns
        average_sales =
        recession_data.groupby('Vehicle_Type')['Automobile_Sales'].mean().reset_index()

        R_chart2 = dcc.Graph(
            figure=px.bar(data_frame=average_sales,
                           x='Vehicle_Type',
                           y='Automobile_Sales',
                           title="The average monthly sales, categorised by vehicle type, during periods of recession."))
        # Plot 3 Pie chart for total expenditure share by vehicle type during recessions
        # grouping data for plotting
        # Hint:Use Vehicle_Type and Advertising_Expenditure columns
        exp_rec=
        recession_data.groupby('Vehicle_Type')['Advertising_Expenditure'].sum().reset_index()
        R_chart3 = dcc.Graph(
            figure = px.pie(data_frame=exp_rec,
                             names='Vehicle_Type',
                             values='Advertising_Expenditure',
                             hover_data=['Advertising_Expenditure'],
                             title="Total Advertising Expenditure Share by Vehicle Type During Recessionary Periods")
        )
        # Plot 4 bar chart for the effect of unemployment rate on vehicle type and sales
        #grouping data for plotting
        # Hint:Use unemployment_rate,Vehicle_Type and Automobile_Sales columns
        unemp_data =
        recession_data.groupby(['Vehicle_Type','unemployment_rate'])['Automobile_Sales'].mean(
        ).reset_index()
        R_chart4 = dcc.Graph(figure=px.bar(data_frame=unemp_data,
                                             x='unemployment_rate',
                                             y='Automobile_Sales',
                                             color='Vehicle_Type',
                                             barmode='group',
                                             labels={'unemployment_rate': 'Unemployment Rate', 'Automobile_Sales': 'Average Automobile Sales'},

```

```

        title='Effect of Unemployment Rate on Vehicle Type and Sales'))
    return [R_chart1, R_chart2, R_chart3, R_chart4 ]
# TASK 2.6: Create and display graphs for Yearly Report Statistics
# Yearly Statistic Report Plots
# Check for Yearly Statistics.
elif (selected_year and selected_statistics== 'Yearly Statistics') :
    yearly_data = data[data['Year'] == selected_year]
    #plot 1 Yearly Automobile sales using line chart for the whole period.
    # grouping data for plotting.
    # Hint:Use the columns Year and Automobile_Sales.
    yas= data.groupby('Month')['Automobile_Sales'].mean().reset_index()
    Y_chart1 = dcc.Graph(figure=px.line(data_frame=yas,
                                       x = 'Month',
                                       y = 'Automobile_Sales',
                                       title = "Total Annual Automobile Sales")
                        )
    # Plot 2 Total Monthly Automobile sales using line chart.
    # grouping data for plotting.
    # Hint:Use the columns Month and Automobile_Sales.
    mas=yearly_data.groupby('Month')['Automobile_Sales'].sum().reset_index()
    Y_chart2 = dcc.Graph(figure=px.line(data_frame=mas,
                                       x='Month',
                                       y='Automobile_Sales',
                                       title='Total Monthly Automobile Sales')
                        )
    # Plot bar chart for average number of vehicles sold during the given year
    # grouping data for plotting.
    # Hint:Use the columns Year and Automobile_Sales
    avr_vdata=yearly_data.groupby('Vehicle_Type')['Automobile_Sales'].mean().reset_index()
    Y_chart3 = dcc.Graph( figure=px.bar(data_frame = avr_vdata,
                                       x='Vehicle_Type',
                                       y='Automobile_Sales',
                                       title='Average Vehicles Sold by Vehicle Type in
                                       the year {}'.format(selected_year)
                                       )
                        )
    # Total Advertisement Expenditure for each vehicle using pie chart
    # grouping data for plotting.
    # Hint:Use the columns Vehicle_Type and Advertising_Expenditure

    exp_data=yearly_data.groupby('Vehicle_Type')['Advertising_Expenditure'].sum().reset_in
dex()
    Y_chart4 = dcc.Graph(
        figure=px.pie(data_frame = exp_data,
                      names = 'Vehicle_Type',
                      values = 'Advertising_Expenditure',
                      title = 'Total Advertisement Expenditure for Each
                      Vehicle in the year {}'.format(selected_year)
                      )
    )

    #TASK 2.6: Returning the graphs for displaying Yearly data
    return [ Y_chart1, Y_chart2, Y_chart3, Y_chart4 ]

else:
    return None

# Run the Dash app
if __name__ == '__main__':
    app.run(debug=False)

```